DE4W Capstone Project

Building a Live Activity Recognition System

Orhan Konak

Table of contents

1 DE4W Capstone Project

2 Welcome

2.1 From Machine Learning to AI Systems

Welcome to the **DE4W Capstone Project**.

This workbook is not only about learning machine learning.

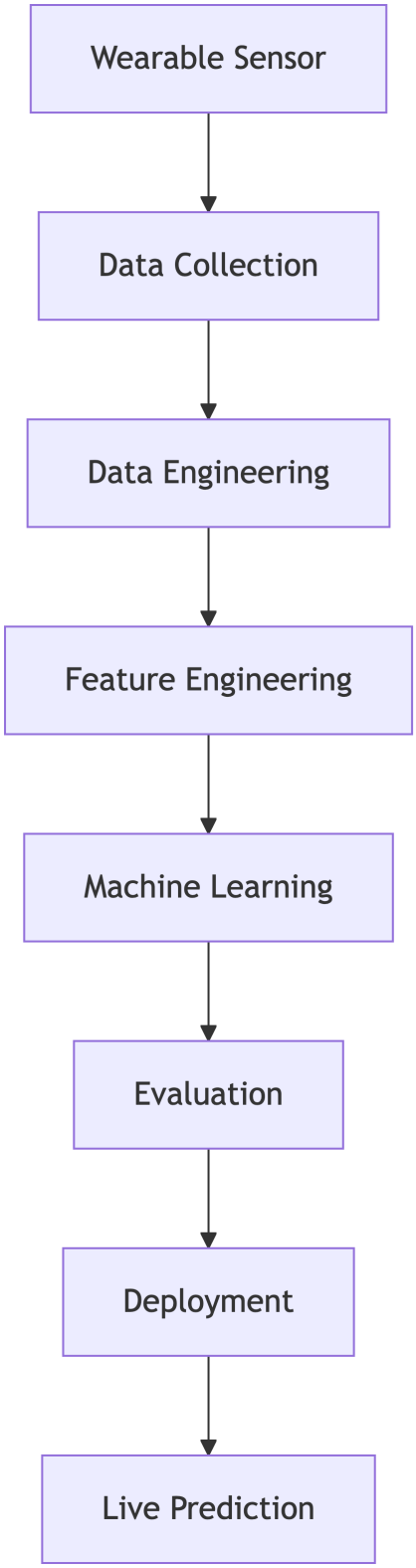
It is about engineering a complete AI system for wearable sensor data.

During the semester, you will connect ideas that are often taught separately: sensors, data collection, data engineering, feature engineering, model training, evaluation, deployment and live prediction.

The result will be an end-to-end wearable AI application that starts with smartphone accelerometer data and ends with real-time activity recognition.

> Engineering Mindset
>
> Every chapter contributes one component of the final system. The goal is to understand how each part works on its own and how the parts fit together.

2.2 Project Pipeline



2.3 Why This Project Matters

Modern AI systems are much more than trained models.

In real projects, data engineering often requires more work than the machine learning algorithm itself. Sensor data must be loaded, cleaned, segmented, transformed and checked before it becomes useful for a model.

Deployment is also part of the work. A model that only runs in a notebook is not yet an application. Real systems need reusable code, model artifacts, APIs, user interfaces and clear error handling.

Reproducibility and software engineering make the project trustworthy. If training and live prediction use different feature logic, the system becomes unreliable. If paths, artifacts or dependencies are unclear, other people cannot run the work.

2.4 What You Will Build This Semester

By the end of this project, you will have built:

- a reusable Python project,
- a data loading and feature engineering pipeline,
- a Random Forest activity classifier,
- subject-wise evaluation for generalization,
- a FastAPI backend for predictions,
- a Streamlit dashboard for live demos,
- live smartphone activity recognition with phyphox.

The system will focus on three activities:

- walking,
- sitting,
- standing.

This small activity set keeps the machine learning task understandable while still showing the full engineering workflow.

2.5 The Journey

The chapters guide you through the system step by step:

1. Introduction and motivation
2. Dataset
3. Loading and exploration
4. Windowing
5. Feature engineering
6. Machine learning
7. Evaluation
8. Deployment
9. Live prediction

By the end of this workbook you will have built a complete end-to-end machine learning application.

3 Introduction and Motivation

> **Why this chapter matters**
>
> This chapter explains why wearable activity recognition is a realistic data engineering problem, not just a machine learning example. It sets up the full path from a Sensor Stream to a Prediction that can support a real application decision.

3.1 Wearables Are Continuous Data Systems

Wearables are everywhere: smartphones, smartwatches, fitness trackers, medical sensors and research devices all measure signals from daily life.

They do not produce neat spreadsheet rows by default.

They produce a continuous Sensor Stream.

Those streams are only the beginning. Raw acceleration values do not directly tell us whether someone is walking, sitting or standing. They are measurements, not yet information.

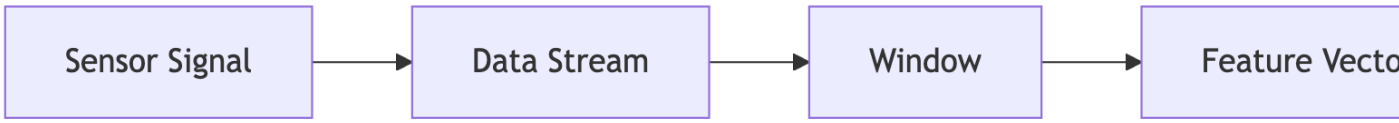
Data engineering is the bridge between raw sensing and meaningful prediction.

This project demonstrates that bridge. We start with wearable sensor signals, transform them into structured data, train and evaluate a model, and finally connect the result to a live application.

> **Engineering Insight**
>
> A good wearable ML system is not defined only by its model accuracy, but by the reliability of the complete pipeline.

3.2 From Sensor to Decision

An activity recognition system turns a physical signal into an application-level decision.



Each step changes the form of the data:

- the sensor measures movement,
- the Sensor Stream preserves measurements over time,
- Windows make the stream manageable,
- Feature Vectors summarize movement patterns,
- the Model maps features to activities,
- the application uses Predictions to support a user-facing task.

3.3 Why Activity Recognition?

Physical activity is clinically and behaviorally relevant.

Movement patterns can help us understand mobility, routines, recovery, fatigue, sedentary behavior and changes in daily life.

Activity recognition is also a classic wearable sensing problem. It is simple enough to teach clearly, but realistic enough to expose the challenges that appear in larger digital health systems.

In this project we focus on walking, sitting and standing.

These activities are simple, but meaningful:

- walking is dynamic and often easier to detect,
- sitting and standing are low-motion states,
- distinguishing sitting from standing shows why wearable ML is not always obvious.

The same pipeline can generalize to more complex tasks, such as rehabilitation monitoring, fall-risk assessment, sleep behavior analysis or long-term mobility tracking.

3.4 The Engineering Challenge

Wearable data is useful because it comes from real life.

That is also what makes it difficult.

Data arrives continuously, so we must decide how to segment it into Windows. Labels are often imperfect because human activity does not always change at clean boundaries. Subjects differ in body movement, walking style and phone handling. Phone placement changes between pockets, hands, bags and tables.

Live systems add another layer of reality. Even if the Model is correct, a demo can fail because the phone and laptop cannot communicate, a network blocks local traffic, phyphox is not reachable or a model artifact is missing.

This is why the project treats software engineering, reproducibility and deployment as part of the learning goal rather than as optional extras.

3.5 Thinking Like An Engineer

Instead of asking:

> What model should I train?

ask:

> What information is available, and what must happen before it can become a reliable Prediction?

This shift matters because the Model is only one component of the final system.

3.6 What Have We Achieved?

- We framed wearable activity recognition as an end-to-end engineering problem.
- We connected raw sensing to application decisions.
- We identified why a Sensor Stream needs structure before it can support a Model.
- We previewed the path from data to Deployment and Live Prediction.

3.7 What Comes Next?

Before building the pipeline, we need to understand the data source. The next chapter introduces the WISDM dataset and explains why it is a good starting point for this project.

4 Understanding the Dataset

> **Why this chapter matters**
>
> Every wearable AI system starts with data, and the data source determines what the system can learn. This chapter introduces the WISDM recordings so we know what kind of Sensor Stream, labels, and subjects our pipeline must handle.

4.1 Learning Objectives

After completing this chapter, you should be able to

- describe what the WISDM dataset contains,
- explain why smartphone accelerometer data is useful for this project,
- interpret one raw sensor measurement,
- explain how activity labels make supervised learning possible,
- justify why we start with walking, sitting and standing.

4.2 Why We Start With The Data

Before we train a model, we need to understand what kind of data we have.

Machine learning does not begin with an algorithm. It begins with measurements, labels, assumptions and constraints.

In this project, the WISDM dataset gives us real smartphone sensor recordings. Each row is one timestamped accelerometer measurement. Activity labels then turn raw movement into supervised learning data.

The goal of this chapter is not to load all files yet. That happens in the next chapter.

The goal here is to understand what the dataset represents and why it is suitable for the capstone project.

Engineering Insight

Dataset choices are engineering decisions. A smaller, well-understood problem is often better for building a reliable first system.

4.3 What Is WISDM?

WISDM is a public activity recognition dataset used in wearable and mobile sensing research.

It contains sensor recordings from smartphones and smartwatches across multiple participants and multiple daily activities.

This makes it useful for a reproducible teaching project:

- the data comes from real devices,
- the labels are already available,
- the task is understandable,
- the same files can be used by every student,
- the dataset is rich enough to demonstrate a complete pipeline.

For this capstone, we use the smartphone accelerometer recordings.

4.4 Why Phone Accelerometer Data?

Almost every smartphone contains an accelerometer.

That makes phone accelerometer data a good first example for wearable data engineering: it is familiar, accessible and directly connected to movement.

It also matches the live demo later in the project. The phyphox app streams smartphone acceleration from a real phone, so the offline WISDM data and the live demo use the same basic sensor type.

Accelerometer signals contain enough variation to demonstrate the full pipeline:

- dynamic movement during walking,
- lower movement during sitting,
- subtle differences between sitting and standing,

- subject-to-subject variation.

4.5 One Row, One Measurement

Each raw WISDM row represents one timestamped sensor measurement.

The raw format is:

```
subject_id, activity_code, timestamp, x, y, z;
```

Column	Meaning	Why it matters
`subject_id`	Participant identifier	Needed for subject-wise evaluation
`activity_code`	Letter code for the activity	Becomes the supervised learning label
`timestamp`	Time of the measurement	Preserves sequence and sampling behavior
`x`	Acceleration along the x-axis	One dimension of phone movement
`y`	Acceleration along the y-axis	One dimension of phone movement
`z`	Acceleration along the z-axis	One dimension of phone movement

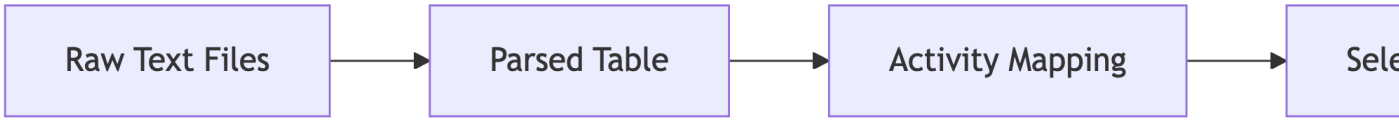
The x, y and z values are not yet features for a classifier.

They are raw signal measurements that we will later transform into Windows and Feature Vectors.

4.6 From Files To A Learning Table

The dataset begins as raw text files.

The project turns those files into a structured table for analysis and model training.



Activity labels are essential because they connect movement measurements to human meaning.

Without labels, we would only have sensor values.

With labels, the model can learn relationships between movement patterns and activities.

4.7 Why Only Three Activities?

WISDM contains many activities, but we intentionally start with three:

- walking,
- sitting,
- standing.

This keeps the first system simple enough to understand.

Walking is dynamic and usually produces a strong movement signal. Sitting and standing are intentionally similar low-motion activities. That similarity makes the task realistic: not every useful classification problem is visually obvious from the raw signal.

The goal is the full engineering pipeline, not maximum dataset complexity.

Once the pipeline works, the same structure can be extended to more activities or more sensors.

4.8 Thinking Like An Engineer

Instead of asking:

> How many activities can this dataset support?

ask:

> Which activities make the first reliable end-to-end system easiest to understand?

A smaller, well-understood problem helps us build trust in every step from Sensor Stream to Prediction.

4.9 Checkpoint

Before continuing, you should be able to answer:

1. What does one row in the raw dataset represent?
2. Why do activity labels matter for supervised learning?
3. Why is smartphone accelerometer data a good first wearable signal?
4. Why do we start with only walking, sitting and standing?

4.10 Further Exploration

Optional: look up other activities available in WISDM.

Which activities would you expect to be easy to distinguish from walking?

Which activities might be confused with sitting or standing?

4.11 What Have We Achieved?

- We identified WISDM as a reproducible dataset for wearable activity recognition.
- We connected each row to one timestamped accelerometer measurement.
- We explained how labels turn raw movement into supervised learning data.
- We justified starting with walking, sitting, and standing.

4.12 What Comes Next?

Now that we know what data we have, we need to check whether it behaves as expected. The next chapter loads the dataset and explores its quality before any Model is trained.

5 Loading and Exploring the Data

Why this chapter matters

Before a Sensor Stream becomes a Feature Matrix, we need to check whether the data is usable. This chapter teaches the habit that protects every later result: validate and understand the dataset before trusting a Model.

5.1 Never Trust Your Data

Every machine learning project starts with a quiet but important rule:

> Never trust your data.

This does not mean we assume the dataset is bad.

It means we verify what is actually there before we build anything on top of it.

Models only learn from the data we give them. If the data is incomplete, mislabeled, imbalanced, noisy or structured differently than we expect, the model will learn those problems too.

Tables help us count things.

Plots help us see things.

Both matter.

Visualization often reveals problems that are invisible in a summary table: spikes, flat signals, strange transitions, sensor noise or activity patterns that look less distinct than expected.

Engineering Insight

Experienced ML engineers often spend much more time exploring and validating data than training models. A strong model cannot rescue a weak understanding of the dataset.

5.2 Setup

We use reusable project functions to load the dataset.

The important part of this chapter is not the Python syntax.

The important part is what we check and why.

```
from pathlib import Path
import sys

import matplotlib.pyplot as plt
import pandas as pd

current_path = Path.cwd().resolve()

for parent in [current_path] + list(current_path.parents):
    if (parent / "app").exists():
        PROJECT_ROOT = parent
        break
else:
    raise RuntimeError("Could not find project root containing app/ directory.")

sys.path.insert(0, str(PROJECT_ROOT))

from app.config import DATA_DIR
from app.data_loader import TARGET_ACTIVITIES, load_wisdm_phone_accel
from app.pipeline import add_magnitude
```

```
df = load_wisdm_phone_accel(
    DATA_DIR,
    target_activities=TARGET_ACTIVITIES,
)

df = add_magnitude(df)
```

5.3 Why Explore Data Before Building Models?

Before training, we need to ask:

- What did we actually load?
- Which subjects are represented?
- Which activities are represented?
- How many samples do we have?
- Do the signals look physically plausible?
- Are there obvious problems that would make evaluation misleading?

Bad data creates bad models.

Sometimes the problem is dramatic, such as missing files.

Sometimes it is subtle, such as one activity being recorded much more often than another, or one subject contributing far more data than the rest.

A professional data engineer does not wait for the model to fail.

They inspect the data first.

5.4 What Does Our Dataset Look Like?

This first table answers the basic question:

> What kind of dataset are we about to use?

```
dataset_summary = pd.DataFrame(
    [
        {
            "question": "How many samples?",
            "answer": len(df),
            "engineering meaning": "More samples can help, but only if they represent the rea
        },
        {
            "question": "How many subjects?",
            "answer": df["subject_id"].nunique(),
            "engineering meaning": "More subjects make generalization to new people more real
        },
        {
            "question": "Which activities?",
            "answer": ", ".join(sorted(df["activity"].unique())),
            "engineering meaning": "The model can only learn activities present in the data.'
        },
```

```
        {
            "question": "Which signal dimensions?",
            "answer": "x, y, z, magnitude",
            "engineering meaning": "These are the raw movement signals we will later summari
        },
    ]
)

dataset_summary
```

	question	answer	engineering meaning
0	How many samples?	814013	More samples can help, but only if they repres...
1	How many subjects?	51	More subjects make generalization to new peopl...
2	Which activities?	sitting, standing, walking	The model can only learn activities present in...
3	Which signal dimensions?	x, y, z, magnitude	These are the raw movement signals we will lat...

Every number tells us something.

The number of samples tells us the scale of the data.

The number of subjects tells us whether we can test generalization across people.

The activity list tells us what the model can and cannot learn.

The signal dimensions tell us what physical measurements are available.

Now we check whether the classes are roughly balanced.

```
activity_counts = (
    df["activity"]
    .value_counts()
    .rename_axis("activity")
    .reset_index(name="samples")
)

activity_counts
```

	activity	samples
0	walking	279817
1	standing	269604
2	sitting	264592

This table answers:

> Is one activity dominating the dataset?

If one class is much larger than the others, accuracy can become misleading. A model might perform well overall while still doing poorly on the smaller classes.

We also check the number of samples per subject.

```
subject_counts = (
    df.groupby("subject_id")
    .size()
    .describe()
    .to_frame(name="samples per subject")
)

subject_counts
```

	samples per subject
count	51.000000
mean	15961.039216
std	6580.439581
min	10717.000000
25%	10719.000000
50%	13532.000000
75%	21477.500000
max	27294.000000

This table answers:

> Do subjects contribute similar amounts of data?

Large differences matter because models may become biased toward subjects with more recordings.

5.5 Looking At The Signals

Sensor values are physical measurements.

They are not abstract numbers.

A phone accelerometer measures acceleration along three axes: x, y and z.

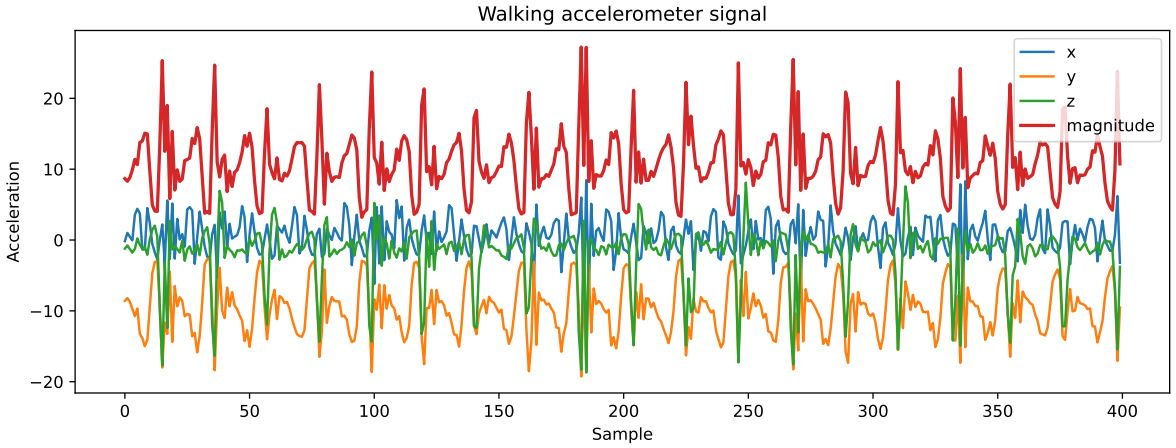
The signal includes several effects:

- gravity,
- dynamic acceleration from movement,
- noise,
- variability from phone placement and subject behavior.

We first inspect one walking segment.

```
walking_sample = (
    df[df["activity"] == "walking"]
    .sort_values("timestamp")
    .head(400)
    .reset_index(drop=True)
)

plt.figure(figsize=(12, 4))
plt.plot(walking_sample["x"], label="x")
plt.plot(walking_sample["y"], label="y")
plt.plot(walking_sample["z"], label="z")
plt.plot(walking_sample["magnitude"], label="magnitude", linewidth=2)
plt.title("Walking accelerometer signal")
plt.xlabel("Sample")
plt.ylabel("Acceleration")
plt.legend()
plt.show()
```



How should we read this figure?

The individual axes depend on how the phone is oriented.

Gravity contributes to the signal even when the phone is not moving.

Walking adds dynamic acceleration, so the signal changes repeatedly over time.

The magnitude line combines x, y and z into one movement-strength signal. It is not perfect, but it is often easier to interpret than any single axis.

5.6 What Can We Already Observe?

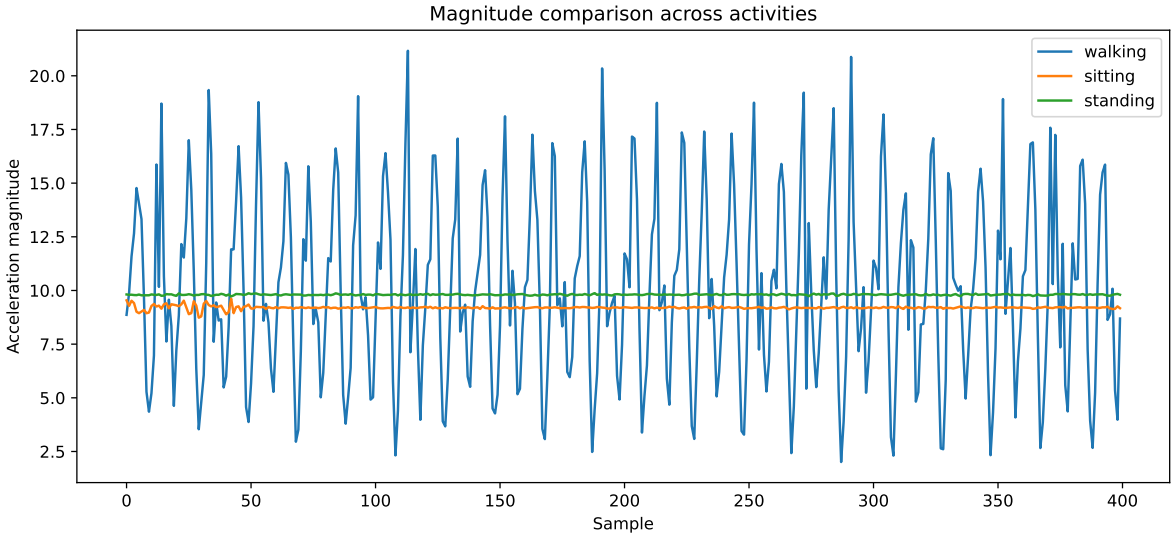
Now we compare the three target activities.

```
plt.figure(figsize=(12, 5))

for activity in TARGET_ACTIVITIES:
    signal = (
        df[df["activity"] == activity]["magnitude"]
        .head(400)
        .reset_index(drop=True)
    )
    plt.plot(signal, label=activity)

plt.title("Magnitude comparison across activities")
plt.xlabel("Sample")
```

```
plt.ylabel("Acceleration magnitude")
plt.legend()
plt.show()
```



This plot is not a model.

It is an engineering sanity check.

Ask yourself:

- Can you already see why walking may be easier to classify?
- Do sitting and standing look more similar?
- Are there spikes or unusual regions?
- Does the signal look plausible for human movement?

Walking usually has stronger and more repeated movement.

Sitting and standing are lower-motion states. They may be meaningful in real life, but they are harder to separate from accelerometer data alone.

That observation will matter later when we evaluate the model.

5.7 Sampling Rate Sanity Check

The WISDM documentation states that the data was collected at approximately 20 Hz.

We should still verify that assumption from the timestamps.

```
subject = df["subject_id"].iloc[0]
activity = "walking"

sample = df[
    (df["subject_id"] == subject)
    & (df["activity"] == activity)
].sort_values("timestamp")

dt = sample["timestamp"].diff().dropna()
estimated_sampling_rate = 1e9 / dt.mean()

estimated_sampling_rate
```

```
np.float64(19.85939378331699)
```

This number answers:

> How much real time does one sample represent?

That matters because window size is measured in samples.

A 100-sample window means something different at 20 Hz than it would at 100 Hz.

5.8 Data Quality Checklist

Before modeling, a data engineer should build a checklist.

Check	Why it matters
Missing values	Missing x, y or z values can break feature extraction or create misleading statistics.
Unexpected timestamps	Irregular timing changes the meaning of a fixed-size window.

Check	Why it matters
Constant signals	A flat signal may indicate a sensor problem or a recording that contains no useful movement.
Sensor spikes	Extreme values can dominate features such as maximum and energy.
Different recording lengths	Some subjects or activities may influence the model more than others.
Class imbalance	Accuracy can hide poor performance on smaller classes.
Subject imbalance	Evaluation can become biased toward heavily represented participants.

This checklist is not bureaucracy.

It protects the rest of the pipeline.

Every later step depends on the assumptions we accept here.

5.9 Thinking Like An Engineer

A beginner might ask:

> Can I train a model?

An engineer asks:

> What assumptions am I making about the data?

For this dataset, we are already making several assumptions:

- activity labels are correct enough to learn from,
- the sampling rate is close enough to 20 Hz,
- windows will capture meaningful movement,
- the selected activities are represented well enough,
- subject differences are important and should be evaluated later.

The point of exploration is to make those assumptions visible.

Once assumptions are visible, we can test them.

5.10 Checkpoint

Before continuing, you should be able to answer:

1. Why should we inspect data before training a model?
2. Why can visualization reveal problems that tables miss?
3. Why might walking be easier to classify than sitting or standing?
4. Why does subject imbalance matter?
5. Why is the sampling rate important for windowing?

5.11 Further Exploration

Optional investigations:

- compare the same activity for two different subjects,
- compare signal histograms across activities,
- look for unusually flat or spiky signal segments.

These are not required for the first pass.

They are examples of how professional data exploration can go deeper.

5.12 What Have We Achieved?

- We loaded the selected WISDM activities into a usable table.
- We inspected subjects, activities, samples, and signal dimensions.
- We interpreted accelerometer plots instead of trusting tables alone.
- We built a checklist for common Sensor Stream quality problems.
- We practiced asking what assumptions the data supports.

5.13 What Comes Next?

Exploration shows us that the data is continuous, but machine learning needs fixed-size observations. The next chapter turns the Sensor Stream into Windows.

6 Windowing

> **Why this chapter matters**
>
> Wearable sensors produce a Sensor Stream that keeps arriving over time, while classical machine learning expects finite examples. Windowing creates the fixed-size Window objects that make the rest of the pipeline possible.

6.1 How Can A Model Understand Something That Never Stops?

Wearable sensors do not wait for us to ask a neat question.

They keep measuring.

A phone accelerometer can produce sample after sample while a person walks, sits, stands, turns, pauses or changes phone position.

Machine learning models, however, usually expect finite observations: one row, one Feature Vector, one Prediction.

Windowing is the bridge between continuous time and tabular machine learning.

It turns an endless Sensor Stream into many fixed-size pieces that a Model can learn from.

6.2 Setup

We use reusable project code to load the data and create windows.

The purpose of this chapter is to understand the engineering decision behind windowing.

```
from pathlib import Path
import sys

import matplotlib.pyplot as plt

current_path = Path.cwd().resolve()

for parent in [current_path] + list(current_path.parents):
    if (parent / "app").exists():
        PROJECT_ROOT = parent
        break
else:
    raise RuntimeError("Could not find project root containing app/ directory.")

sys.path.insert(0, str(PROJECT_ROOT))

from app.config import WINDOW_SIZE
from app.data_loader import load_wisdm_phone_accel
from app.pipeline import add_magnitude
from app.windowing import create_grouped_windows, create_windows

DATA_DIR = PROJECT_ROOT / "data"

df = load_wisdm_phone_accel(DATA_DIR)
df = add_magnitude(df)
```

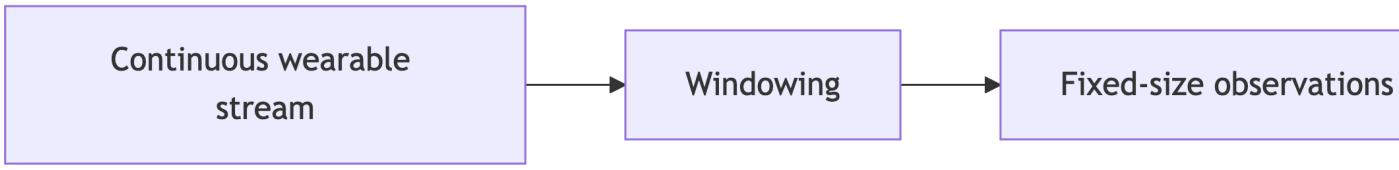
6.3 Continuous Signals vs Machine Learning

A Sensor Stream is like a movie.

A machine learning table is more like a photo album.

The movie never really stops. The table needs individual examples.

That mismatch is why windowing exists.



The stream preserves time.

The window creates a bounded observation.

The model receives a finite object it can classify.

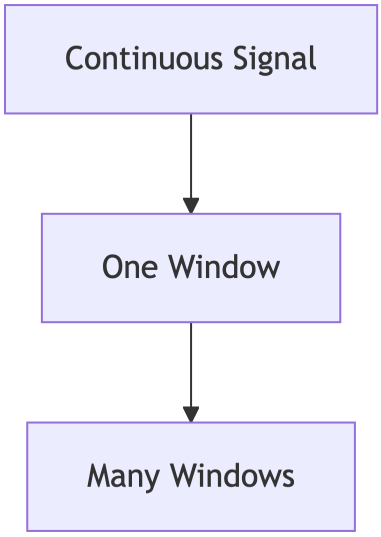
Without windowing, a classical machine learning model has no clear answer to a basic question:

> Which part of the stream should become one training example?

6.4 What Is A Window?

A window is a short segment of consecutive sensor samples.

For example, if the sensor records at about 20 Hz, then 100 samples represent about five seconds.



We first inspect windows for walking.

```
walking_df = (
    df[df["activity"] == "walking"]
    .sort_values("timestamp")
    .copy()
)

walking_windows = create_windows(
    walking_df,
    window_size=WINDOW_SIZE,
)

len(walking_windows)
```

```
2798
```

Each window has the same number of rows.

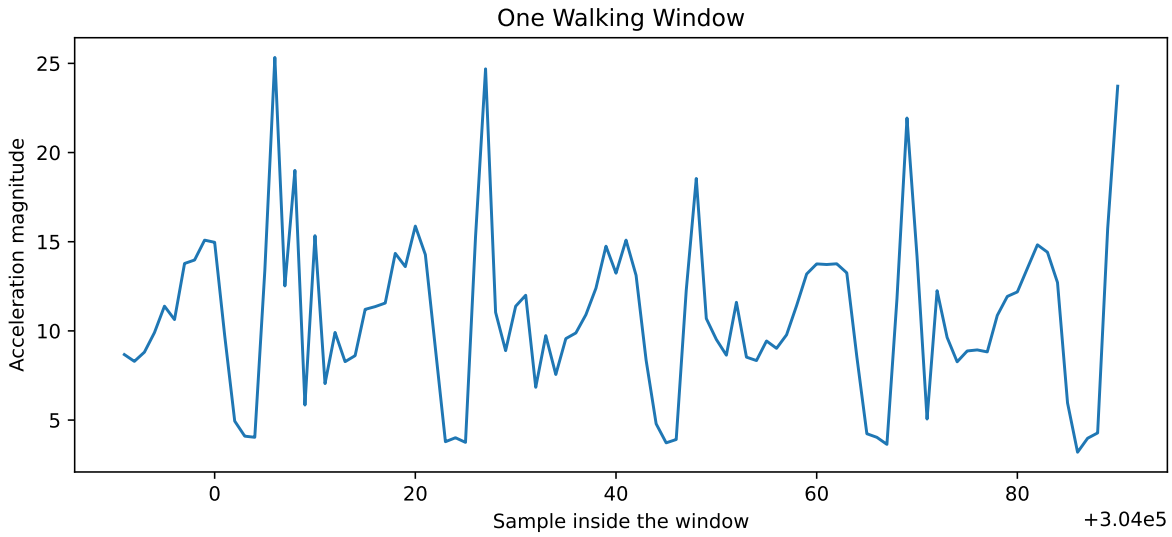
```
first_window = walking_windows[0]

first_window.shape
```

```
(100, 8)
```

Now we visualize one real window.

```
plt.figure(figsize=(10, 4))
plt.plot(first_window["magnitude"])
plt.title("One Walking Window")
plt.xlabel("Sample inside the window")
plt.ylabel("Acceleration magnitude")
plt.show()
```



This figure shows one bounded moment of activity.

The model will not see the entire recording at once.

It will see many windows like this, each summarized into features in the next chapter.

6.5 Why Five Seconds?

This project uses 100 samples per window.

At approximately 20 Hz, that is about five seconds.

Five seconds is not magic.

It is a trade-off.

Short windows can react quickly. That is useful for live prediction because the dashboard can update sooner. But short windows may not contain enough movement to represent the activity clearly.

Long windows are often more stable. They contain more evidence. But they increase latency because the system must wait longer before making a prediction.

For live systems, this trade-off is very concrete:

- shorter window: faster response, noisier prediction,
- longer window: slower response, more stable prediction.

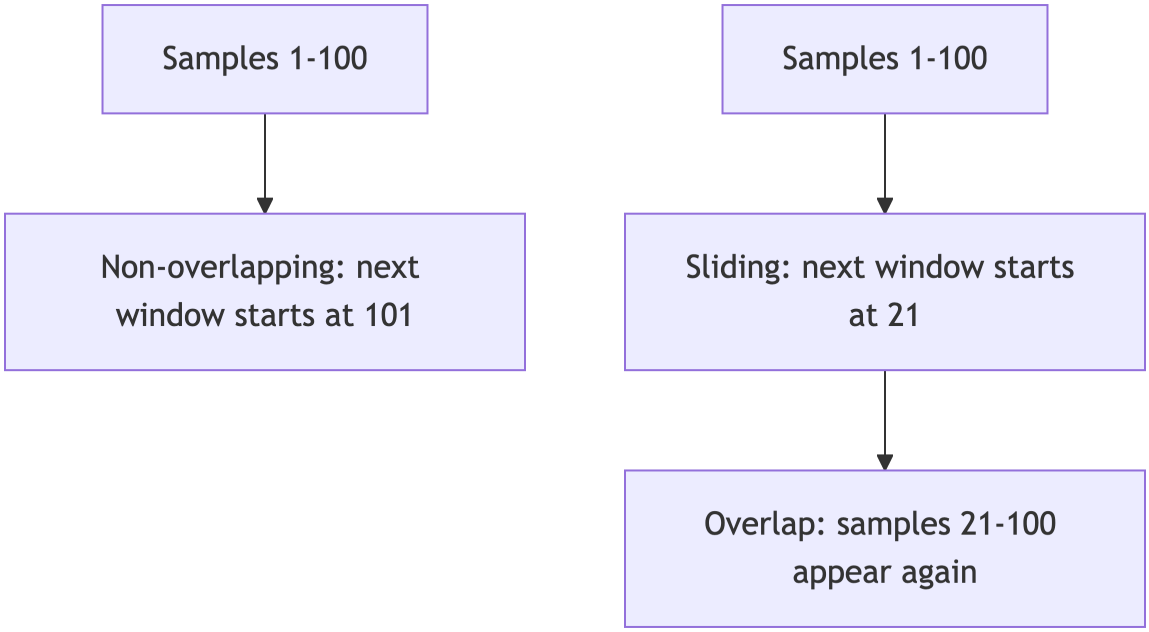
Window size controls how quickly the system can notice that something changed.

6.6 Non-Overlapping vs Sliding Windows

There are two common ways to create windows.

Non-overlapping windows use each sample once.

Sliding windows move forward by a smaller step and overlap with previous windows.



Overlapping windows can produce smoother predictions because the model receives updates more often.

They also duplicate information.

That increases computational cost and can make evaluation more complicated if highly similar windows appear in both train and test data.

This project intentionally starts with non-overlapping windows.

That choice keeps the first system easier to reason about:

- fewer windows,
- less duplicated information,

- simpler evaluation,
- clearer connection between raw data and training examples.

6.7 Choosing A Window Size

Different applications need different windows.

Window Size	Advantages	Disadvantages	Typical Use Cases
1 s	Very responsive	Often noisy and incomplete	Fast gestures, quick feedback
2 s	Still responsive, slightly more context	May miss slower activity patterns	Lightweight live demos
5 s	Good balance of context and latency	Slower to react than short windows	Walking/sitting/standing baseline
10 s	More stable summaries	High latency, delayed feedback	Slow activities, offline analysis

The table does not tell us the universally correct answer.

It tells us what trade-offs we are accepting.

> Engineering Insight
>
> There is no perfect Window size.
> The correct Window depends on the activity, latency requirements, battery, computational resources and deployment scenario.

6.8 Thinking Like An Engineer

Instead of asking:

> What window size is best?

ask:

What trade-offs does my application require?

For a classroom live demo, responsiveness matters because students should see predictions change.

For a clinical monitoring system, stability and reliability might matter more than instant updates.

For a battery-powered wearable, computation and energy use matter too.

Windowing is not just preprocessing.

It is an engineering decision.

6.9 The Project Implementation

The reusable windowing logic lives in `app.windowing`.

This chapter does not redefine that logic.

It imports:

- `create_windows()` for one sorted signal segment,
- `create_grouped_windows()` for safe windowing by subject and activity.

```
all_windows = create_grouped_windows(
    df,
    window_size=WINDOW_SIZE,
)

len(all_windows)
```

```
8066
```

Creating windows separately by subject and activity matters.

Otherwise, one window could accidentally contain multiple people or multiple labels.

That would create training examples that do not mean what we think they mean.

6.10 Checkpoint

Before continuing, you should be able to answer:

1. Why do wearable streams need to be segmented before classical ML?
2. What does a five-second window mean at 20 Hz?
3. Why can short windows be more responsive but less stable?
4. Why can overlapping windows make predictions smoother?
5. Why does this project start with non-overlapping windows?

6.11 Further Exploration

Optional ideas:

- try a smaller window size,
- try a larger window size,
- design an overlapping-window version and discuss what would change.

These are experiments, not required steps.

6.12 What Have We Achieved?

- We explained why continuous Sensor Streams must become finite Windows.
- We compared short, long, overlapping, and non-overlapping Window choices.
- We connected Window size to latency, stability, and Live Prediction.
- We used the reusable `app.windowing` logic instead of duplicating pipeline code.

6.13 What Comes Next?

A Window still contains many raw measurements. The next chapter turns each Window into a compact Feature Vector that a Model can use.

7 Feature Engineering

> **Why this chapter matters**
>
> Windows organize time, but they are still raw measurements. Feature engineering turns each Window into a Feature Vector, which is the bridge between physical motion and a machine learning Model.

7.1 How Can A Computer Recognize Walking Without Ever Seeing A Person?

A computer does not see a person walking.

It sees numbers.

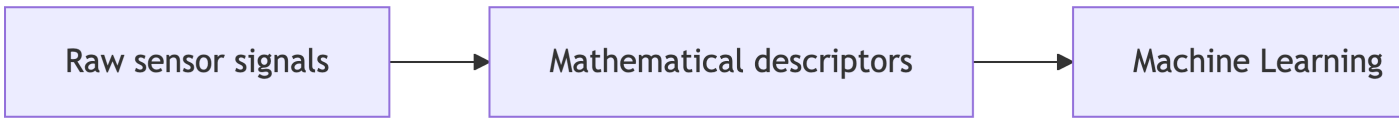
In this project, those numbers come from accelerometer Windows: x, y, z and magnitude values sampled over time.

The central question of this chapter is:

> How can raw sensor signals become something a machine learning model can understand?

The answer is feature engineering.

Feature engineering transforms raw physical measurements into mathematical descriptions of behavior.



7.2 Setup

We use reusable project code for loading, windowing and feature extraction.

The important idea is not the syntax.

The important idea is how motion becomes numbers that can be compared.

```
from pathlib import Path
import sys

import matplotlib.pyplot as plt
import pandas as pd

current_path = Path.cwd().resolve()

for parent in [current_path] + list(current_path.parents):
    if (parent / "app").exists():
        PROJECT_ROOT = parent
        break
else:
    raise RuntimeError("Could not find project root containing app/ directory.")

sys.path.insert(0, str(PROJECT_ROOT))

from app.config import WINDOW_SIZE
from app.data_loader import load_wisdm_phone_accel
from app.feature_extraction import extract_features
from app.pipeline import add_magnitude, windows_to_features
from app.windowing import create_windows

DATA_DIR = PROJECT_ROOT / "data"

df = load_wisdm_phone_accel(DATA_DIR)
df = add_magnitude(df)
```

7.3 Why Raw Signals Are Difficult For Classical ML

A window still contains many measurements.

With 100 samples and four signals, one window contains 400 raw values.

That is a lot of detail, but not yet much meaning.

Classical machine learning models such as Random Forests usually work best with structured feature tables: one row per example, one column per meaningful descriptor.

An analogy:

Imagine describing a song to someone.

You could send every audio sample.

Or you could describe tempo, loudness, rhythm and energy.

The second description loses some detail, but it captures useful structure.

Feature engineering does the same for sensor windows.

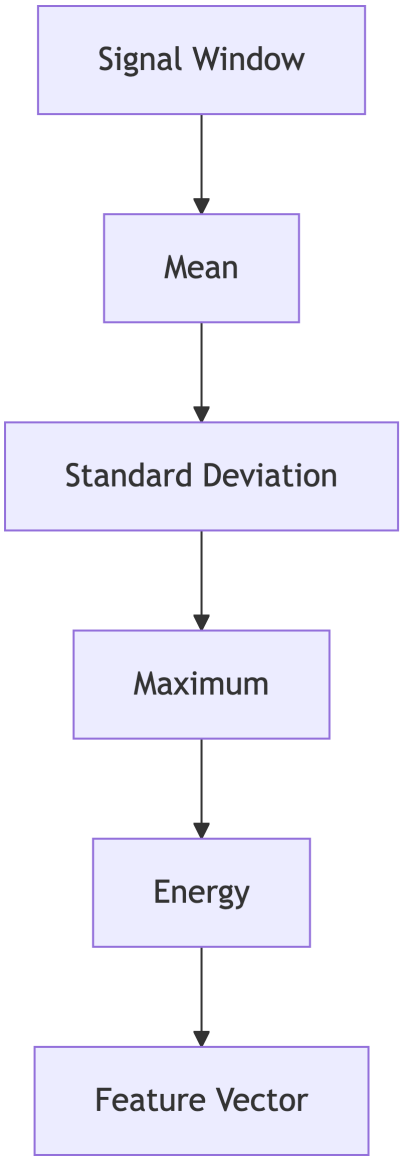
It summarizes behavior instead of asking the model to memorize every sample.

7.4 What Is A Feature?

A feature is not just another measurement.

A feature is information extracted from many measurements.

For one signal window, we can compute descriptors such as average level, variability, extremes and movement intensity.



The feature vector is the compact mathematical description that the model sees.

The raw signal is still important.

But the model receives the summary.

> **Engineering Insight**
>
> Feature engineering is knowledge compression.
> Hundreds of measurements become a compact mathematical representation. Good Feature Vectors preserve information that helps distinguish activities.

7.5 Our Feature Set

For each signal, we compute the same set of descriptors.

Signals:

```
x, y, z, magnitude
```

Features:

Feature	What it describes	Why it helps
mean	Average signal level	Captures orientation and baseline acceleration
standard deviation	Variability	Higher for dynamic movement
minimum	Lowest observed value	Captures one side of the movement range
maximum	Highest observed value	Captures extreme movement
energy	Sum of squared values	Captures overall movement intensity

This gives:

$$4 \times 5 = 20$$

features per window.

This table answers:

> What information might distinguish one activity from another?

Walking should usually have more variability and higher energy than sitting or standing.

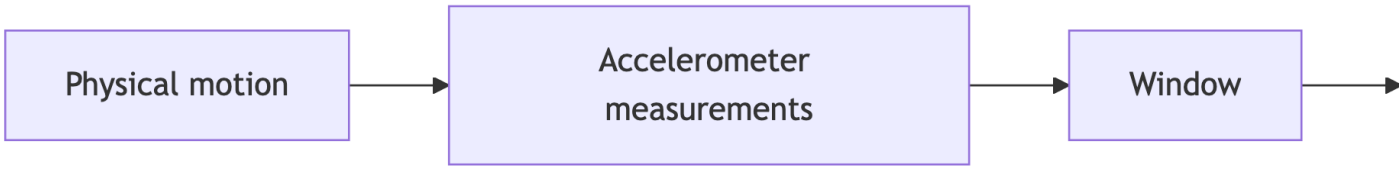
Sitting and standing may have similar energy, which is why they can be harder to separate.

7.6 From Physics To Mathematics

Accelerometers measure physical motion.

Feature engineering converts that motion into numbers.

Machine learning compares numerical patterns.



This is one of the central ideas of the project.

The model does not know what walking looks like.

It learns that certain numerical patterns tend to correspond to walking, sitting or standing.

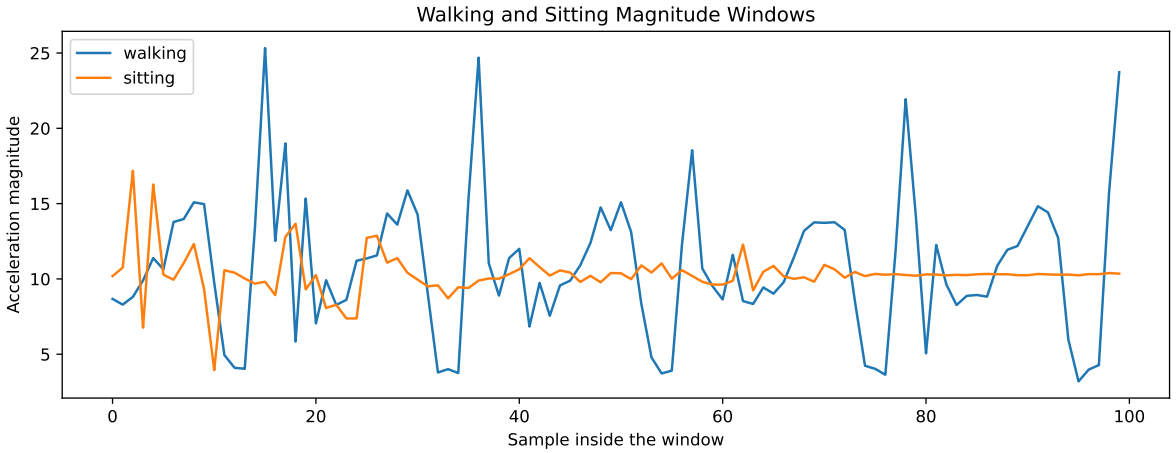
7.7 Why Does Walking Have Higher Energy?

We compare a walking window with a sitting window.

```
walking_window = create_windows(
    df[df["activity"] == "walking"].sort_values("timestamp"),
    window_size=WINDOW_SIZE,
)[0]

sitting_window = create_windows(
    df[df["activity"] == "sitting"].sort_values("timestamp"),
    window_size=WINDOW_SIZE,
)[0]
```

```
plt.figure(figsize=(12, 4))
plt.plot(walking_window["magnitude"].reset_index(drop=True), label="walking")
plt.plot(sitting_window["magnitude"].reset_index(drop=True), label="sitting")
plt.title("Walking and Sitting Magnitude Windows")
plt.xlabel("Sample inside the window")
plt.ylabel("Acceleration magnitude")
plt.legend()
plt.show()
```



What should you observe?

Walking usually produces repeated changes in acceleration magnitude.

Sitting is often flatter because the phone moves less.

Energy is computed as:

$$Energy = \sum_i x_i^2$$

Squaring makes larger movements count more.

So a dynamic activity like walking often produces higher energy than a static activity like sitting.

Now we inspect the actual feature values.

```
comparison = pd.DataFrame(
    [
        {"activity": "walking", **extract_features(walking_window)},
        {"activity": "sitting", **extract_features(sitting_window)},
    ]
)

comparison[
    [
        "activity",
        "std_magnitude",
        "max_magnitude",
        "energy_magnitude",
    ]
]
```

	activity	std_magnitude	max_magnitude	energy_magnitude
0	walking	4.566283	25.331639	13653.06148
1	sitting	1.491087	17.177679	10810.68680

This table answers:

> Do the mathematical descriptors reflect what we saw in the plot?

The point is not that one feature solves the whole problem.

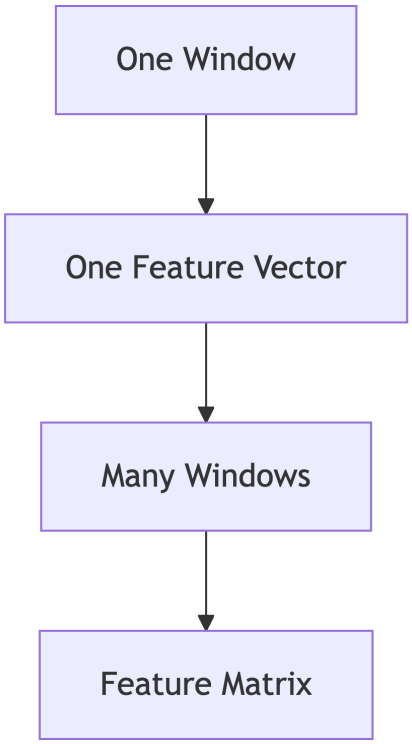
The point is that features translate visual movement patterns into numbers.

7.8 Building A Feature Table

One window becomes one feature vector.

Many windows become a feature matrix.

That matrix is what the machine learning model uses.



The feature table has one row per window.

Feature columns describe movement.

Metadata columns keep the activity label and subject identifier.

```
features_df = windows_to_features(
    df,
    window_size=WINDOW_SIZE,
)

features_df.head()
```

	mean_x	std_x	min_x	max_x	energy_x	mean_y	std_y	min_y	max_y	ene
0	3.845516	0.351159	2.627441	4.488892	1491.007201	4.827262	0.436838	4.073685	5.965424	234
1	4.069495	0.031640	3.964447	4.144730	1656.178255	4.641167	0.033818	4.552872	4.714188	215
2	4.098965	0.035909	3.981873	4.173141	1680.278813	4.643016	0.034376	4.534317	4.727310	215
3	4.150756	0.041034	4.053879	4.252579	1723.044535	4.669522	0.044667	4.566193	4.779709	218
4	4.135481	0.049281	3.972107	4.287186	1710.460832	4.656202	0.051068	4.526337	4.816910	216

```
feature_table_summary = pd.DataFrame(
    [
        {
            "question": "How many windows?",
            "answer": len(features_df),
            "why it matters": "Each window becomes one training example.",
        },
        {
            "question": "How many feature columns?",
            "answer": len([c for c in features_df.columns if c not in ["activity", "subject_
            "why it matters": "This is the numerical description the model receives.",
        },
        {
            "question": "How many subjects?",
            "answer": features_df["subject_id"].nunique(),
            "why it matters": "Subject IDs support realistic evaluation later.",
        },
        {
            "question": "Missing feature values?",
            "answer": int(features_df.isna().sum().sum()),
            "why it matters": "Missing values can break training or distort results.",
        },
    ]
)

feature_table_summary
```

	question	answer	why it matters
0	How many windows?	8066	Each window becomes one training example.
1	How many feature columns?	20	This is the numerical description the model re...
2	How many subjects?	51	Subject IDs support realistic evaluation later.
3	Missing feature values?	0	Missing values can break training or distort r...

This table answers:

> Is the feature table ready to become a machine learning dataset?

7.9 The Project Implementation

The reusable feature extraction logic lives in `app.feature_extraction`.

This chapter does not redefine it.

It imports `extract_features()` and uses `windows_to_features()` from the project pipeline.

That matters because training and live prediction must use the same feature logic.

If the model is trained with one feature definition and deployed with another, predictions become unreliable.

7.10 Thinking Like An Engineer

Instead of asking:

> Which feature should I compute?

ask:

> What information distinguishes one activity from another?

For walking, useful information may include movement intensity, rhythm and variability.

For sitting versus standing, the difference may be smaller and may depend on phone placement.

Feature engineering is where domain knowledge enters the machine learning pipeline.

7.11 Save The Feature Table

The next chapters can load the feature table directly.

Saving it makes the pipeline easier to reproduce and debug.

```
features_df.to_csv(
    DATA_DIR / "features_df.csv",
    index=False,
)
```

7.12 Checkpoint

Before continuing, you should be able to answer:

1. Why are raw windows difficult for classical machine learning?
2. Why is a feature more than one measurement?
3. Why might walking have higher energy than sitting?
4. What does one row in the feature table represent?
5. Why must training and deployment use the same feature extraction logic?

7.13 Further Exploration

Optional feature ideas:

- frequency-domain features,
- entropy,
- jerk,
- correlation between axes.

These can be useful later, but they are not required for the first reliable system.

7.14 What Have We Achieved?

- We explained why raw Windows are difficult for classical machine learning.
- We described how features summarize motion across many measurements.
- We connected physical acceleration to mathematical descriptors.
- We built a Feature Matrix where each row represents one Window.
- We reused `app.feature_extraction` for the project implementation.

7.15 What Comes Next?

Now that we have a Feature Matrix and labels, we can train a Model. The next chapter explains how classical machine learning turns Feature Vectors into Predictions.

8 Machine Learning

> **Why this chapter matters**
>
> Feature engineering gives us numbers; machine learning turns those numbers into activity Predictions. This chapter explains how a Random Forest Model learns patterns in the Feature Matrix without needing to understand human movement.

8.1 How Can A Computer Distinguish Walking From Standing Using Only Numbers?

In the previous chapter, we transformed raw sensor Windows into Feature Vectors.

That was the key step.

The model does not receive a video of a person walking. It does not understand posture, balance or intent.

It receives numbers such as mean, standard deviation, maximum and energy.

The central question of this chapter is:

> How can a computer distinguish walking from standing using only numbers?

The answer is pattern learning.

Feature engineering and machine learning are inseparable.

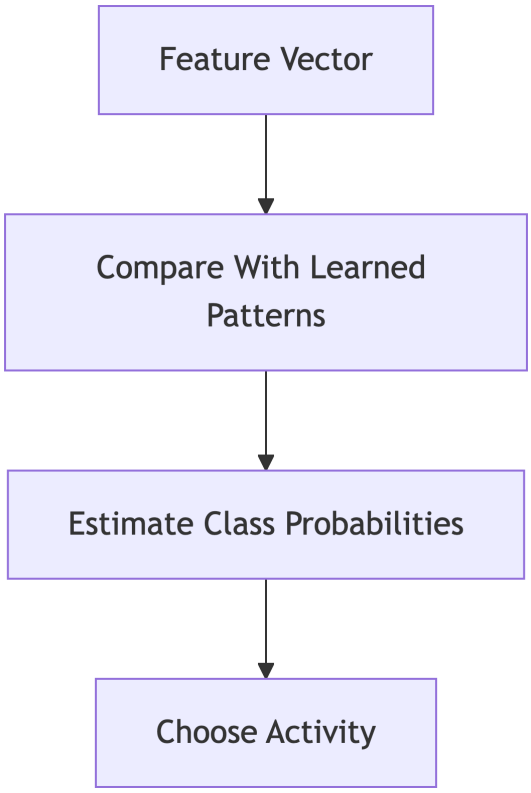
The model can only learn patterns that are visible in the representation we give it.

8.2 From Features To Decisions

Feature Vectors are mathematical descriptions of Windows.

Machine learning searches for patterns in those descriptions.

Prediction is a comparison between a new Feature Vector and patterns learned from labeled examples.



For example, a walking window may have higher magnitude variability and higher energy.

A standing window may have lower variability.

The model learns statistical regularities like these.

It does not learn human meaning directly.

> Engineering Insight
>
> Models learn patterns, not meaning.
> The Model has no understanding of human movement. It only learns statistical regularities in the Feature Matrix.

8.3 Why Classical Machine Learning?

For this project, classical machine learning is the right starting point.

It is transparent enough to teach.

It trains quickly.

It works with relatively little data.

It fits naturally with tabular feature engineering.

It is easy to export and deploy.

Deep learning can be powerful, especially for large raw-signal datasets. But it often requires more data, more compute and more careful tuning. It also makes the first engineering story harder to see.

Here, we want students to understand the complete pipeline.

Classical ML keeps the model simple enough that the system around it remains visible.

8.4 Why Random Forest?

Random Forest is a good baseline for engineered wearable features.

Think of it as a committee of experts.

Each expert is a decision tree.

A decision tree asks a sequence of simple questions:

- Is energy high?
- Is variability low?
- Is the maximum magnitude above a threshold?
- Does this feature pattern look more like walking or sitting?

One tree can be fragile.

It may focus too much on one pattern.

A Random Forest trains many different trees and lets them vote.

That gives us:

- diversity, because different trees see different parts of the data,
- majority vote, because no single tree decides alone,
- robustness, because mistakes from one tree can be balanced by others,
- strong performance on tabular feature data.

This is why Random Forest is a practical first model for this capstone.

8.5 What Does The Model Actually Learn?

The model never truly learns walking, standing or sitting.

It learns numerical regions in feature space.

Conceptually, it learns rules like:

```
high energy + high variability -> often walking
low energy + low variability -> often sitting or standing
similar low-motion patterns -> harder to separate
```

This is not a perfect map.

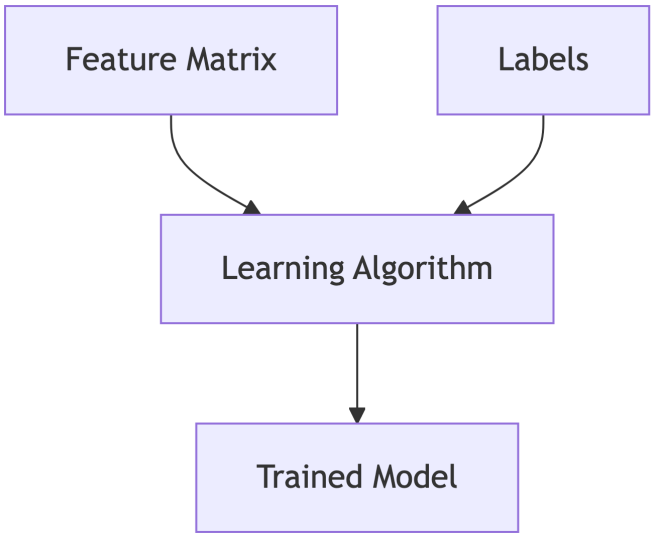
Activities can overlap.

Some sitting windows may look similar to standing windows. Some quiet walking windows may look less dynamic than expected. Some subjects move differently from others.

That overlap is why evaluation matters.

8.6 Training

Training connects feature vectors with labels.



The feature matrix contains one row per window.

The labels tell the model which activity each row represents.

The learning algorithm searches for patterns that connect feature values to labels.

The trained model stores those learned patterns so it can make predictions for new windows.

8.7 Why Can Models Make Mistakes?

Models make mistakes when the numerical patterns are not clear enough.

In wearable activity recognition, this can happen for several reasons.

Sitting and standing overlap because both are low-motion activities.

Measurements are noisy because sensors are imperfect.

Subjects differ because people walk, sit and stand in different ways.

Phone placement changes because a phone may be in a pocket, hand, bag or on a table.

Features may be insufficient because simple statistics do not capture every useful pattern.

Mistakes are not just model failures.

They often reveal something important about the data representation.

8.8 Thinking Like An Engineer

Instead of asking:

> Which model is best?

ask:

> Does my representation contain enough information?

If the features do not distinguish sitting from standing, a more complex model may not fix the problem.

If the data does not represent new subjects well, high accuracy on familiar subjects may be misleading.

Model choice matters.

But representation, data quality and evaluation matter just as much.

8.9 The Project Implementation

The reusable training logic lives in `app.training`.

It handles the project-level steps:

- loading the feature table,
- separating features, labels and subject identifiers,
- training the Random Forest baseline,
- saving model artifacts for later evaluation and deployment.

This chapter does not duplicate that implementation.

The important idea is that training consumes the feature table created in the previous chapter.

8.10 Further Exploration

Optional models to explore later:

- Gradient Boosting,
- XGBoost,
- LightGBM,
- Deep Learning.

These are useful next steps, but they are not required for the first reliable system.

8.11 Checkpoint

Before continuing, you should be able to answer:

1. Why does the model need engineered features?
2. Why is Random Forest a good baseline for tabular features?
3. What does it mean to say the model learns numerical regions?
4. Why can sitting and standing be confused?
5. Why are feature engineering and model training inseparable?

8.12 What Have We Achieved?

- We connected Feature Vectors to activity Predictions.
- We explained why classical machine learning is a strong baseline for this project.
- We described Random Forests as a robust committee of decision trees.
- We clarified that the Model learns numerical patterns, not human meaning.
- We identified why Models can still make mistakes.

8.13 What Comes Next?

Training a Model is not enough. The next chapter asks whether the Prediction will still work for people the Model has never seen before.

9 Evaluation And Generalization

> **Why this chapter matters**
>
> Evaluation tells us whether a Model is ready for the world it will actually face. In wearable sensing, the crucial question is whether Predictions generalize beyond the people whose data appeared during training.

9.1 Does Our Model Really Work On People It Has Never Seen Before?

Evaluation is not just computing a score.

Evaluation is asking whether the system will work in the real world.

For wearable activity recognition, the central question is often:

> Does our Model really work on people it has never seen before?

A Model can look excellent under a random split and still fail to generalize to new subjects.

That gap is not an embarrassment.

It is one of the most important learning outcomes in wearable machine learning.

> **Engineering Insight**
>
> If the system will be used on new people, the evaluation must test new people.

9.2 Setup

We use the reusable evaluation functions from `app.evaluation`.

The goal of this chapter is to interpret the evaluation strategies, not to duplicate their implementation.

```
from pathlib import Path
import sys

import pandas as pd

current_path = Path.cwd().resolve()

for parent in [current_path] + list(current_path.parents):
    if (parent / "app").exists():
        PROJECT_ROOT = parent
        break
else:
    raise RuntimeError("Could not find project root containing app/ directory.")

sys.path.insert(0, str(PROJECT_ROOT))

from app.config import WINDOW_SIZE
from app.evaluation import (
    evaluate_random_split,
    evaluate_subject_split,
    leave_one_subject_out_evaluation,
    metrics_for_metadata,
)
from app.training import (
    load_feature_table,
    save_model_artifacts,
    split_features_and_labels,
    train_random_forest,
)
```

```
features_df = load_feature_table()
X, y, groups, feature_columns = split_features_and_labels(features_df)
```

9.3 Why Evaluation Is Not Just Accuracy

Accuracy is useful.

It gives one simple number: the fraction of predictions that were correct.

But accuracy can also be misleading.

The meaning of an accuracy score depends on how the test data was chosen.

In wearable sensing, deployment often means using the system with new people. That means the real evaluation question is not only:

> Can the model classify held-out rows?

It is:

> Can the model classify activities for subjects it did not see during training?

The difference between those two questions is the heart of this chapter.

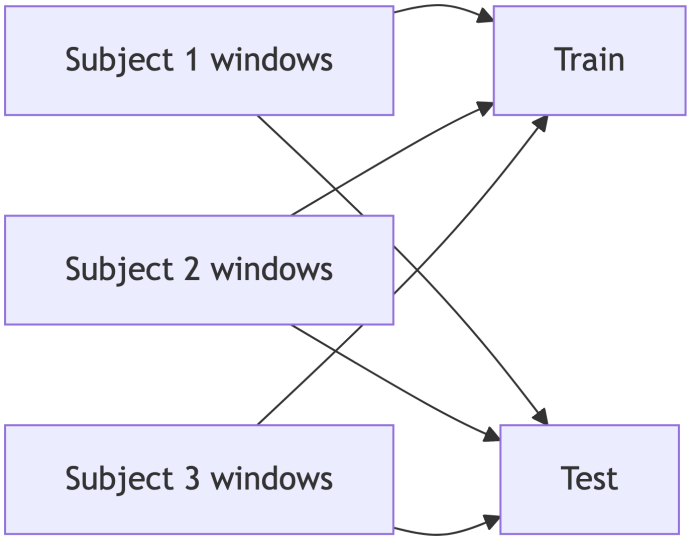
9.4 Random Split: The First Result

A random split divides rows into train and test sets.

It is common because it is simple and works well for many tabular datasets.

But wearable datasets contain many windows from the same subjects.

Random splitting can place windows from one subject in both training and test.



This often gives high performance because the test data is very similar to the training data.

That can be a useful first result.

But for wearable data, it may be too optimistic.

```
random_result = evaluate_random_split(X, y)

random_result["metrics"]["accuracy"]
```

```
0.9857496902106567
```

This score answers:

> Can the model classify unseen windows from a familiar subject pool?

That is not the same as deployment to new people.

9.5 The Leakage Problem

Windows from the same person can look very similar.

People differ in walking style, phone placement, body movement and recording conditions.

If the same subject appears in both train and test, the model may learn subject-specific patterns.

This is not intentional cheating.

It is a subtle evaluation design problem.

> ⚠ Common Pitfall
>
> High accuracy can come from evaluating on data that is too similar to the training data.

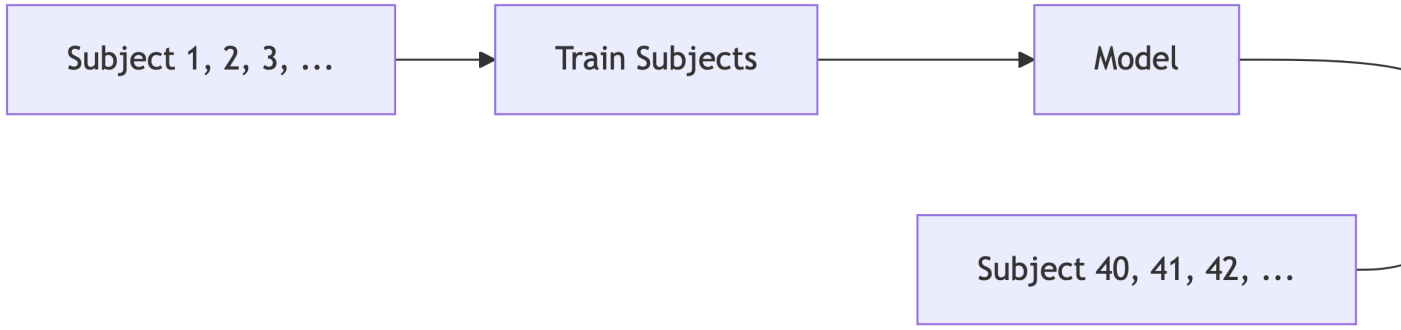
The important question is not whether the random split score is wrong.

The important question is what it actually measures.

9.6 Subject-Wise Evaluation

In subject-wise evaluation, train subjects and test subjects are separated.

The model trains on one group of people and is tested on different people.



This better simulates deployment on a new user.

Accuracy often drops.

That drop is informative, not a failure.

It tells us how much harder generalization is when the model cannot rely on familiar subject patterns.

```
subject_result = evaluate_subject_split(X, y, groups)

subject_result["metrics"]["accuracy"]
```

```
0.8678756476683938
```

This score answers:

> Can the model classify activities for people it has never seen before?

For wearable systems, this is often the more important question.

9.7 Random Split Vs Subject-Wise Split

The comparison is the lesson.

```
comparison = pd.DataFrame(
    [
        {
            "evaluation strategy": "Random split",
            "accuracy": random_result["metrics"]["accuracy"],
            "test examples": random_result["test_size"],
        },
        {
            "evaluation strategy": "Subject-wise split",
            "accuracy": subject_result["metrics"]["accuracy"],
            "test examples": subject_result["test_size"],
        },
    ]
)

comparison
```

	evaluation strategy	accuracy	test examples
0	Random split	0.985750	1614
1	Subject-wise split	0.867876	1544

Evaluation strategy	What is tested?	Typical result	Risk
Random split	New windows from a mixed subject pool	Often high	Can be too optimistic if subjects appear in both train and test
Subject-wise split	New subjects held out from training	Often lower	Depends on which subjects are held out
LOSO	Every subject is tested as the held-out subject once	Strict and detailed	Computationally more expensive

If the random split score is higher, do not panic.

That gap tells us that subject generalization is harder than row-level prediction.

9.8 Interpreting Confusion Matrices

A confusion matrix shows which activities are predicted correctly and which are confused.

Diagonal entries are correct predictions.

Off-diagonal entries are mistakes.

```
random_labels = random_result["metrics"]["labels"]
random_matrix = pd.DataFrame(
    random_result["metrics"]["confusion_matrix"],
    index=[f"true_{label}" for label in random_labels],
    columns=[f"pred_{label}" for label in random_labels],
)

random_matrix
```

	pred_sitting	pred_standing	pred_walking
true_sitting	511	9	4
true_standing	2	527	6
true_walking	2	0	553

This matrix tells us where the random split model succeeds and fails.

Walking is often easier because it has stronger movement and higher energy.

Sitting and standing may be confused because both are low-motion states.

Now inspect the subject-wise confusion matrix.

```
subject_labels = subject_result["metrics"]["labels"]
subject_matrix = pd.DataFrame(
    subject_result["metrics"]["confusion_matrix"],
    index=[f"true_{label}" for label in subject_labels],
    columns=[f"pred_{label}" for label in subject_labels],
)

subject_matrix
```

	pred_sitting	pred_standing	pred_walking
true_sitting	467	42	7
true_standing	154	359	1
true_walking	0	0	514

This matrix is more realistic for deployment.

If sitting and standing are confused more often here, that is not surprising.

It means the distinction is less stable across people.

9.9 LOSO Concept

Leave-One-Subject-Out evaluation is stricter.

The idea is simple:

1. Hold out one subject.
2. Train on all other subjects.
3. Test on the held-out subject.
4. Repeat for each subject.
5. Average the performance.

LOSO asks:

How well does the system generalize across all possible held-out subjects?

For a teaching render, we run a small preview on the first five subjects.

```
loso_preview = leave_one_subject_out_evaluation(
    X,
    y,
    groups,
    max_subjects=5,
)

loso_preview
```

	subject_id	accuracy	test_size
0	1600	1.000000	105
1	1601	1.000000	135
2	1602	1.000000	105
3	1603	0.963504	137
4	1604	0.961905	105

This preview shows that subject-level performance can vary.

Some people are easier for the model.

Some people are harder.

That variation is exactly why subject-aware evaluation matters.

9.10 Thinking Like An Engineer

Instead of asking:

How high is my accuracy?

ask:

Does my evaluation simulate the way the system will be used?

If the system will be used by new people, then new people must appear in the test set.

If the system will be used on a new device, evaluation should eventually test device differences.

If the system will be used in a new session, evaluation should consider session differences.

The evaluation design is part of the engineering design.

9.11 The Project Implementation

The reusable evaluation logic lives in `app.evaluation`.

It provides:

- random split evaluation,
- subject-wise evaluation,
- LOSO-style evaluation,
- classification reports,
- confusion matrix support.

This chapter uses those functions so the evaluation logic stays consistent across the project.

9.12 Save Evaluation Metadata

The project saves evaluation results with the model artifacts.

This makes the trained model easier to inspect later.

```
final_model = train_random_forest(X, y)

metadata = {
    "evaluation_results": {
        "random_split": metrics_for_metadata(random_result),
        "subject_split": metrics_for_metadata(subject_result),
        "loso_preview": loso_preview.to_dict(orient="records"),
    }
}
```

```
model_metadata = save_model_artifacts(
    final_model,
    feature_columns,
    metadata=metadata,
    window_size=WINDOW_SIZE,
)

{
    "model_type": model_metadata["model_type"],
    "window_size": model_metadata["window_size"],
    "class_labels": model_metadata["class_labels"],
}
```

```
{'model_type': 'RandomForestClassifier',
 'window_size': 100,
 'class_labels': ['sitting', 'standing', 'walking']}
```

9.13 Further Exploration

Optional directions:

- stratified LOSO,
- session-wise split,
- device-wise split,
- calibration,
- confidence thresholds.

These are advanced evaluation ideas, not required for the first system.

9.14 Checkpoint

Before continuing, you should be able to answer:

1. Why can random split accuracy be too optimistic?
2. Why does subject-wise evaluation better match deployment on new people?

3. What does LOSO test?
4. How do you read diagonal and off-diagonal confusion matrix entries?
5. Why should evaluation match the deployment scenario?

9.15 What Have We Achieved?

- We explained why accuracy depends on the deployment scenario.
- We identified how random splits can overestimate wearable sensing performance.
- We separated subject-wise evaluation from ordinary random evaluation.
- We interpreted confusion matrices as evidence about Model behavior.
- We framed LOSO as a stricter test of generalization to new people.

9.16 What Comes Next?

Once evaluation gives us a trustworthy baseline, the system needs a way to serve Predictions. The next chapter moves from evaluation to Deployment with FastAPI and Streamlit.

10 Deployment

> **Why this chapter matters**
>
> A Model only becomes useful when another program can ask it for a Prediction. This chapter turns the trained artifact into a small Deployment so the wearable AI system can be tested through an API and a dashboard.

10.1 Learning Objectives

After completing this chapter, you should be able to

- explain which model artifacts are needed for prediction,
- start the FastAPI service,
- check API health,
- send one prediction request,
- explain how the Streamlit dashboard uses the same prediction code.

10.2 Model Artifacts

The deployed prediction service uses files in the `models/` directory.

```
models/
   activity_model_random_forest.joblib
   feature_columns.joblib
   activity_labels.joblib
```

The most important artifact for deployment is the feature column list.

The live system must send features to the model in exactly the same order that was used during training.

Engineering Insight

Deployment is part of the machine learning system, not an afterthought.
The same Feature Vector definition must connect training, evaluation, API prediction, and the Streamlit dashboard.

Common Pitfall

If the model was trained with one set of features but the API receives a different set, predictions are not reliable.
The API checks for missing and unexpected features before predicting.

10.3 FastAPI Service

FastAPI exposes the trained model as a small HTTP service.

The project keeps the API logic in `app/api.py` and the model loading/prediction logic in `app/model_utils.py`.

Start the API from the project root:

```
uv run uvicorn app.api:app --reload
```

The API will usually be available at:

```
http://127.0.0.1:8000
```

10.4 Health Check

Use the health endpoint to verify that the API can load the model artifacts.

```
curl http://127.0.0.1:8000/health
```

Expected response:

```
{
  "status": "ok",
  "model_loaded": true,
  "detail": "Model artifacts are available."
}
```

If a model file is missing, the endpoint returns a clear message naming the missing artifact.

10.5 Prediction Request

The prediction endpoint is:

```
POST /predict
```

It expects one JSON object with a `features` field.

The feature names must match `models/feature_columns.joblib`.

Example shape:

```
{
  "features": {
    "mean_x": 0.0,
    "std_x": 0.0,
    "min_x": 0.0,
    "max_x": 0.0,
    "energy_x": 0.0
  }
}
```

The real request must include all 20 trained feature columns.

The response contains:

```
{
  "activity": "walking",
  "confidence": 0.91,
  "probabilities": {
    "sitting": 0.02,
    "standing": 0.07,
    "walking": 0.91
  }
}
```

> **Validation**
>
> If features are missing, the API returns a validation error that names them.

10.6 Streamlit Dashboard

The Streamlit dashboard is the live demonstration UI.

Start it from the project root:

```
uv run streamlit run dashboard.py
```

The dashboard:

- reads live samples from phyphox,
- stores recent samples in `LiveBuffer`,
- creates a Window,
- extracts a Feature Vector,
- calls the same prediction service used by the API,
- displays the current activity and probabilities.

The API and dashboard share the same model artifact checks.

This reduces the chance that the web service and live demo behave differently.

10.7 Thinking Like An Engineer

Instead of asking:

> Can my notebook produce a prediction?

ask:

> Can another system request the same Prediction reliably?

Deployment forces us to make assumptions explicit: artifact locations, feature names, input validation, output format, and failure messages.

10.8 Checkpoint

Before continuing, you should be able to answer:

1. Which model artifacts are needed for prediction?
2. Why must feature names match the training feature columns?
3. What does `/health` check?
4. What does `/predict` return?
5. Why should the API and dashboard use the same prediction service?

10.9 Further Exploration

Optional: intentionally remove one feature from a `/predict` request.

Which error message does the API return?

How does that help debug deployment problems?

10.10 What Have We Achieved?

- We identified the model artifacts needed for Deployment.
- We explained why feature columns must match the trained Feature Matrix.
- We started and tested the FastAPI prediction service.
- We connected the API and Streamlit dashboard to the same prediction path.
- We practiced treating validation errors as part of system reliability.

10.11 What Comes Next?

Deployment gives us a service that can return Predictions. The final major chapter connects that service to a real phone Sensor Stream for Live Prediction.

11 Live Prediction

> **Why this chapter matters**
>
> Live Prediction closes the loop between a real phone Sensor Stream and the trained Model. This chapter shows how all earlier components work together when data arrives continuously and the system has to respond in real time.

11.1 Learning Objectives

After completing this chapter, you should be able to

- configure phyphox remote access,
- explain why the phone and laptop must be on the same network,
- describe the live prediction pipeline,
- run the Streamlit dashboard,
- troubleshoot common live demo failures.

11.2 phyphox Setup

Install phyphox on your smartphone.

Open an acceleration experiment and enable remote access.

phyphox will show a URL such as:

```
http://192.168.178.45:8080
```

Enter this URL in the Streamlit dashboard.

> **Network Requirement**
>
> The phone and laptop must be able to reach each other on the local network.
> University WiFi networks such as eduroam often block direct device-to-device communication.
> If the laptop cannot open the phyphox URL in a browser, the dashboard cannot read sensor data either.

Recommended network options:

- smartphone hotspot,
- local home WiFi,
- travel router,
- lab router configured for local device communication.

11.3 Start the Dashboard

From the project root:

```
uv run streamlit run dashboard.py
```

The dashboard lets you configure:

- phyphox URL,
- window size,
- demo duration.

The demo is intentionally time-limited so that it cannot run forever by accident.

11.4 Live Prediction Pipeline

The live pipeline is:

```
phyphox
→ PhyphoxClient
→ LiveBuffer
→ Window
→ Feature Vector
→ prediction service
→ Streamlit dashboard
```

`PhyphoxClient` reads the latest x, y and z acceleration values.

`LiveBuffer` stores the most recent samples until a full Window is available.

The Window is converted to the same Feature Vector used during training.

The prediction service loads the trained Random Forest Model and returns:

- predicted activity,
- confidence,
- class probabilities.

> Engineering Insight
>
> Live Prediction is where hidden assumptions become visible.
> Sampling rate, network access, buffer size, feature order, and model artifacts all have to work at the same time.

11.5 Dashboard States

The dashboard shows the current live demo state:

State	Meaning
Waiting	The demo has not started yet
Collecting samples	The buffer is filling
Predicting	A full window is available
Error	phyphox or model prediction failed
Finished	The configured demo duration ended

These states make live demos easier to debug in front of a class.

11.6 Thinking Like An Engineer

Instead of asking:

> Does the Model work?

ask:

> Can the complete live system keep producing Predictions when real-world conditions change?

In a classroom demo, the most likely failures may be networking, phone setup, timing, or missing artifacts rather than machine learning accuracy.

11.7 Troubleshooting

11.7.1 phyphox URL Does Not Open

Check that remote access is enabled in phyphox.

Make sure the phone and laptop are on the same network.

If you are on eduroam, try a phone hotspot or a local router.

11.7.2 Dashboard Shows a Timeout

The laptop can see the URL but the response is too slow.

Move both devices closer to the access point or use a simpler local network.

11.7.3 Missing Buffer Error

Make sure the correct phyphox experiment is open and producing acceleration buffers named `accX`, `accY` and `accZ`.

11.7.4 Model Artifact Error

Confirm that these files exist:

```
models/activity_model_random_forest.joblib
models/feature_columns.joblib
models/activity_labels.joblib
```

If they are missing, rerun the training/export step before the live demo.

11.8 Checkpoint

Before continuing, you should be able to answer:

1. Why can eduroam block the live demo?
2. What does the live buffer store?
3. When does the dashboard start predicting?
4. Why must live feature extraction match training feature extraction?
5. What should you check first if phyphox cannot connect?

11.9 Further Exploration

Optional: compare predictions with two different window sizes.

How does a smaller window affect responsiveness?

How does a larger window affect stability?

11.10 What Have We Achieved?

- We connected phyphox Sensor Stream data to the project pipeline.
- We traced the live path from `LiveBuffer` to Window to Feature Vector to Prediction.
- We explained why eduroam and other managed networks can block the demo.
- We interpreted dashboard states as debugging signals.
- We prepared a troubleshooting checklist for reliable Live Prediction.

11.11 What Comes Next?

You have now followed the full system from raw Sensor Stream to Live Prediction. The remaining project materials help you review, teach, and verify the complete workbook.